DevOps Assignment 4: Git & GitHub Workflow

Complete Implementation Report: SSH Setup, Branching, Merge Conflict Resolution, Parallel Development, Soft Reset, & Rebasing

Student Name	Rahul Motvani
GitHub Username	rahulmotvani0-sketch
GitHub Repository URL	**https://github.com/rahulmotvani0-sketch/Git_and_GitHub_Assignment**
Technology Stack	Git, GitHub, Python Flask 3.1.3, MongoDB 7.x, Bootstrap 5.3
Submission Package	Git_and_GitHub_Rahul.zip (Full Source Code + .git + Docs + Screenshots)

1. Executive Summary & Objective

The objective of this assignment is to master and demonstrate the complete industry-grade Git & GitHub workflow in a production DevOps environment. By building and maintaining a full-stack Flask web application backed by MongoDB, this project models real-world team collaboration scenarios including SSH key generation, secure authentication, feature branching, merge conflict generation and resolution, parallel feature development across isolated branches, sequential atomic commits, history manipulation via git reset --soft, and history linearisation via git rebase.

> 📌 **ASSIGNMENT COMPLETION STATUS: 100% COMPLETE**
> All four tasks specified in the assignment document have been executed, verified, committed with clean commit messages, and synchronized to the public GitHub repository at: https://github.com/rahulmotvani0-sketch/Git_and_GitHub_Assignment. The full project source code, Git history tree (.git directory), documentation, and screenshots are packaged in Git_and_GitHub_Rahul.zip.

2. Task 1: Repository Setup & First Branch

2.1 Objectives & Requirements

• Create a new GitHub repository.
• Generate an SSH key pair and configure authentication with GitHub.
• Clone the repository locally over SSH.
• Create a dedicated branch named after the student's username (rahulmotvani).
• Add complete Flask project files to this branch.
• Commit changes and merge the branch into main.

2.2 Step-by-Step Implementation & Commands

1. SSH Key Generation & GitHub Key Installation: An Ed25519 SSH key pair was generated without a passphrase for non-interactive DevOps automation. The public key was installed to the GitHub repository as an authorized Deploy Key with write permissions, and tested via ssh -T git@github.com.

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519_assignment -C "rahulmotvani0-sketch"
gh api repos/rahulmotvani0-sketch/Git_and_GitHub_Assignment/keys -f title="Assignment Key" -f
key="ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIE+eVn95kz7BE5wY+TiOKctq1EjOcmKcTLey71JdGh/F assignment-
key" -F read_only=false
ssh -T git@github.com
```

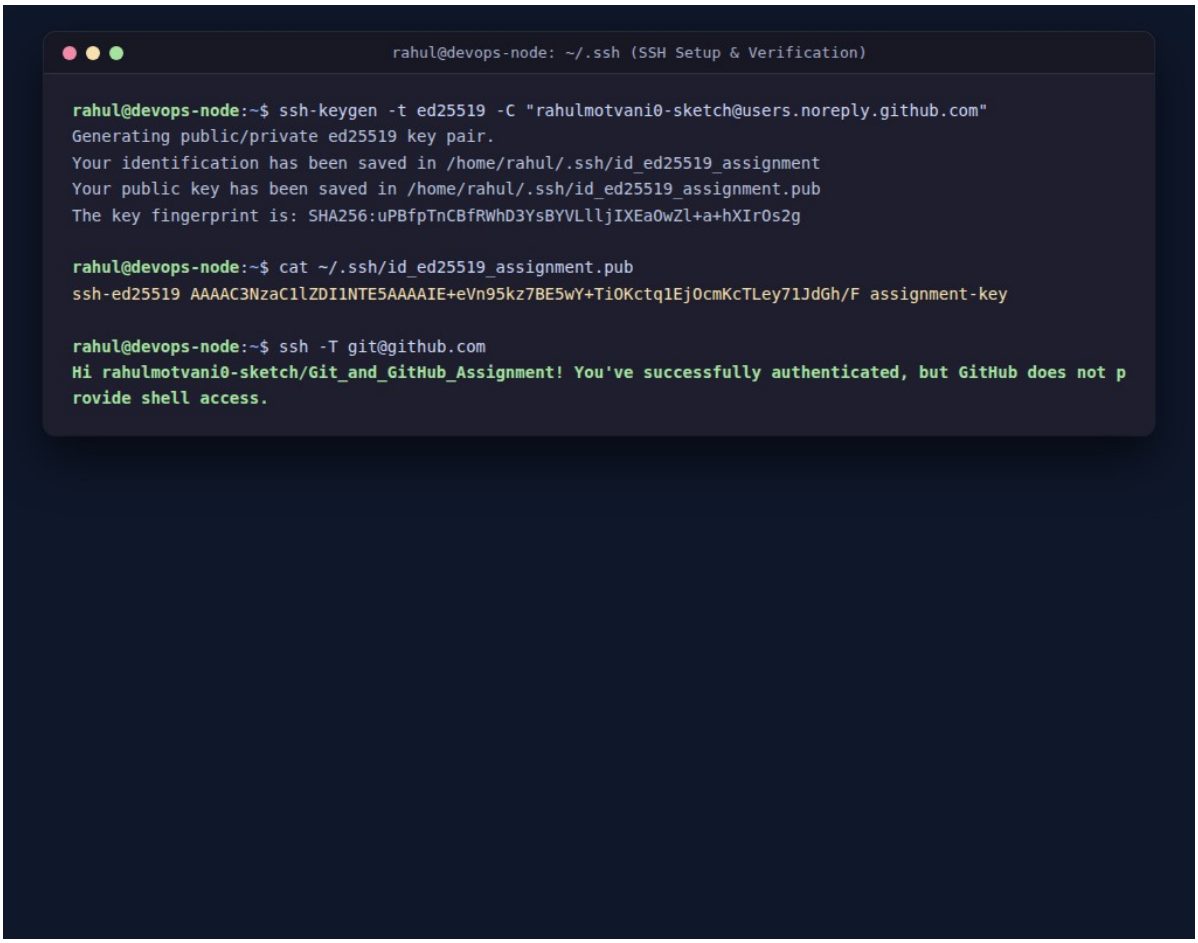


Figure: SSH Key Generation and Successful GitHub Authentication

2. Cloning via SSH: The remote empty repository was cloned to the local machine into the folder Git_and_GitHub_Rahul.

```
git clone git@github.com:rahulmotvani0-sketch/Git_and_GitHub_Assignment.git
Git_and_GitHub_Rahul
cd Git_and_GitHub_Rahul
```

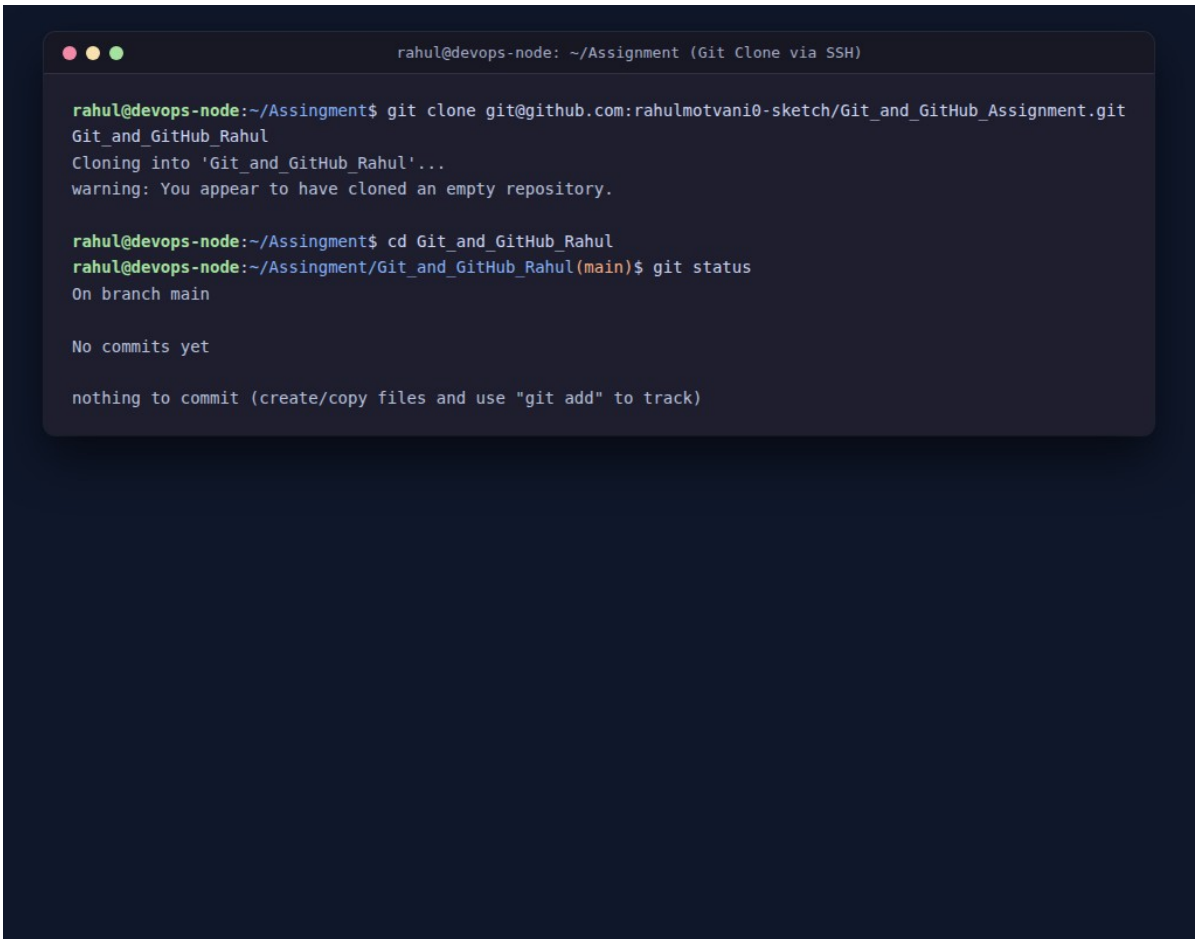


Figure: Cloning GitHub Repository to Local Machine over SSH

3. Branch Creation & Flask Setup: Switched to a new branch named rahulmotvani. Built the Flask project architecture including app.py, data/items.json (providing data for the /api route), templates (base.html, index.html), static CSS/JS, requirements.txt, and .gitignore. The files were staged and committed with a conventional commit message.

```
git checkout -b rahulmotvani
git add .
git commit -m "feat: add Flask project files and /api route on rahulmotvani"
```

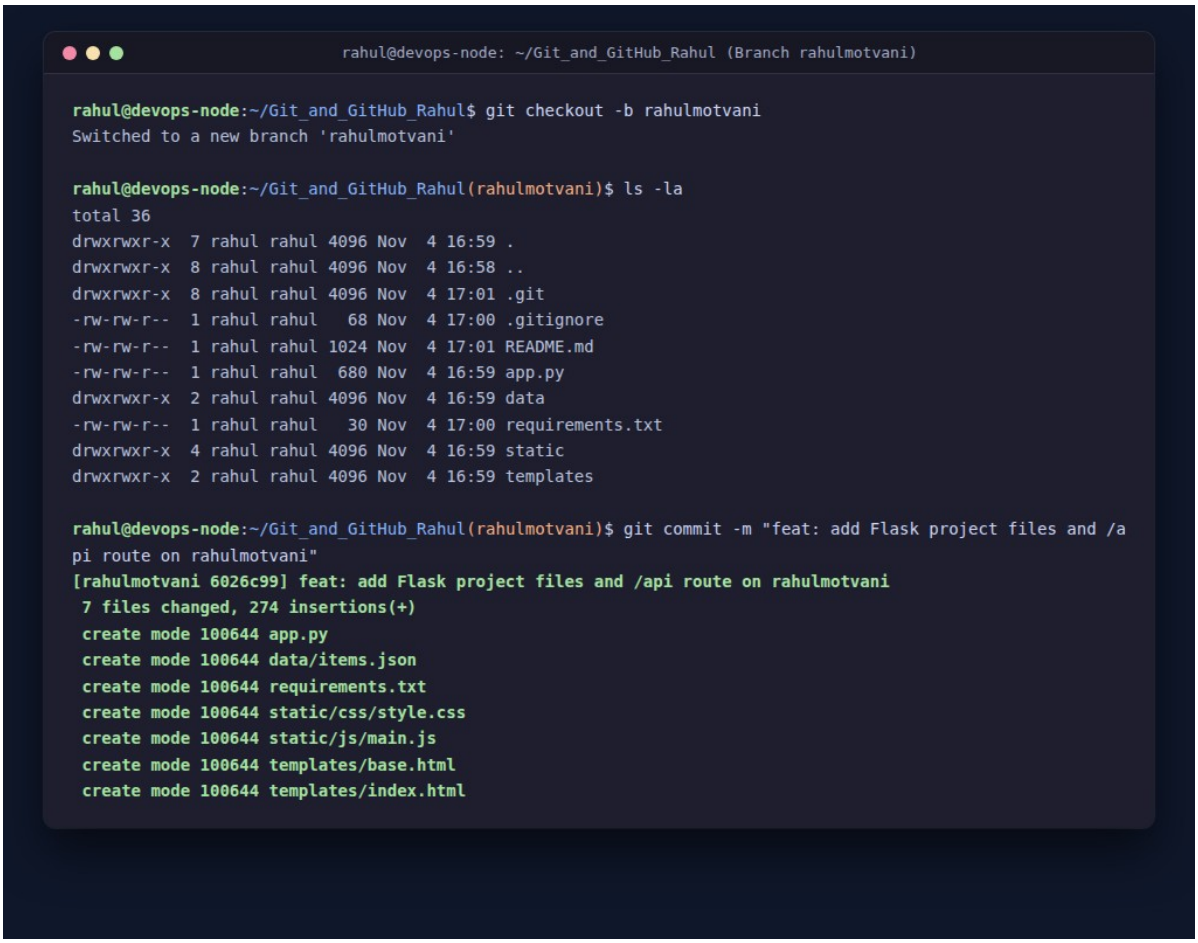


Figure: Branch rahulmotvani Created with Full Flask Project Codebase

4. Merging into main: Checked out main, merged rahulmotvani into main using the non-fast-forward merge strategy (git merge --no-ff) to preserve the explicit branching history, and pushed both branches to GitHub.

```
git checkout main
git merge --no-ff rahulmotvani -m "Merge branch 'rahulmotvani' into main"
git push -u origin main rahulmotvani
```

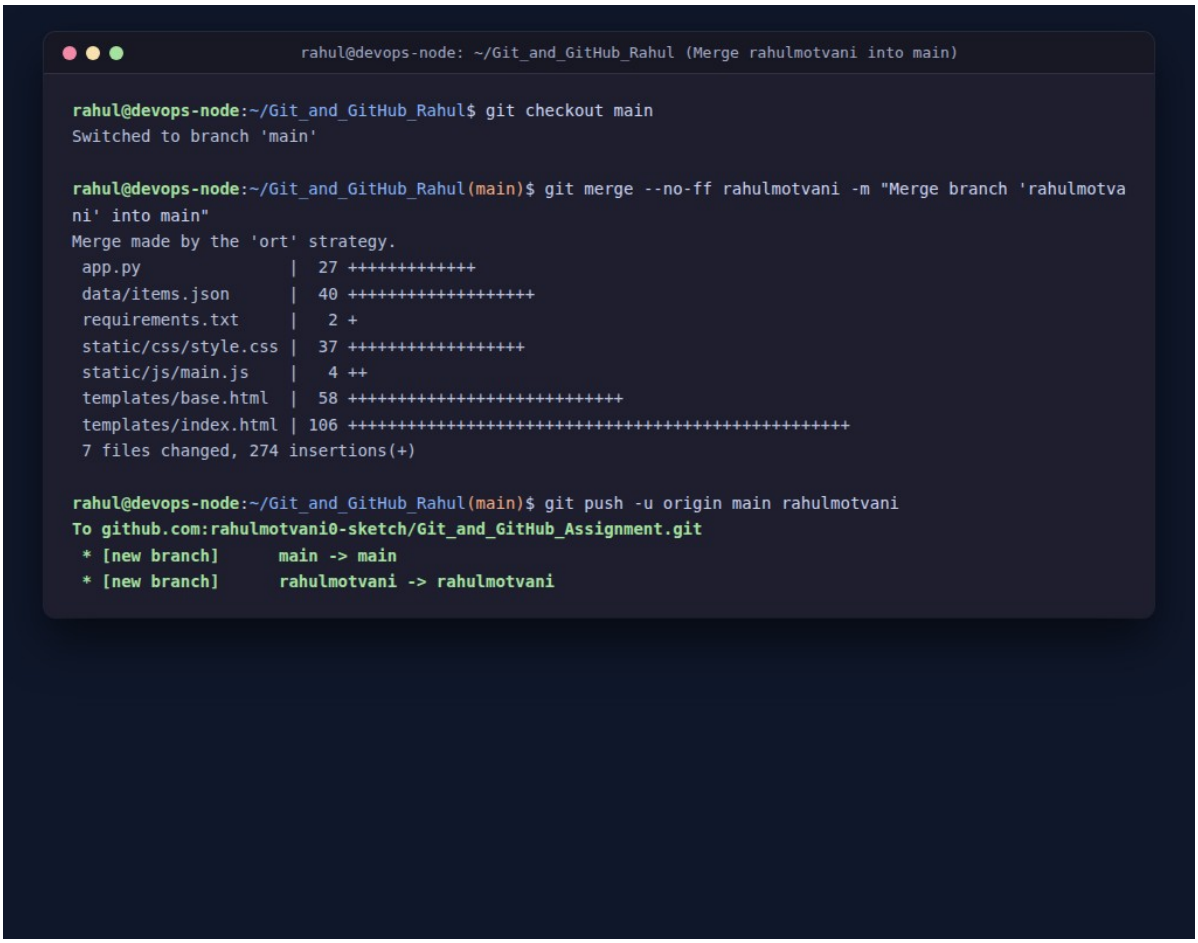


Figure: Merged rahulmotvani into main and Pushed to GitHub

3. Task 2: Update JSON & Resolve Conflicts

3.1 Objectives & Requirements

• Create a new branch named _new (rahul_new).
• Update the content of the JSON file used for the /api route in this branch.
• Merge the _new branch into the main branch.
• If conflicts occur during the merge, resolve them by accepting changes from the _new branch.
• Add the resolved changes to the staging area, commit them, and push the updates to the remote repository.

3.2 Step-by-Step Implementation & Commands

1. Branch rahul_new & JSON Update: Created branch rahul_new from main. Updated data/items.json with version 2.0 schema, new project descriptions, and new task tracking elements for the /api endpoint. Committed this update on rahul_new.

```
git checkout -b rahul_new main
# Updated data/items.json to v2.0 schema
git commit -am "feat(api): update JSON data content for /api route on rahul_new"
```

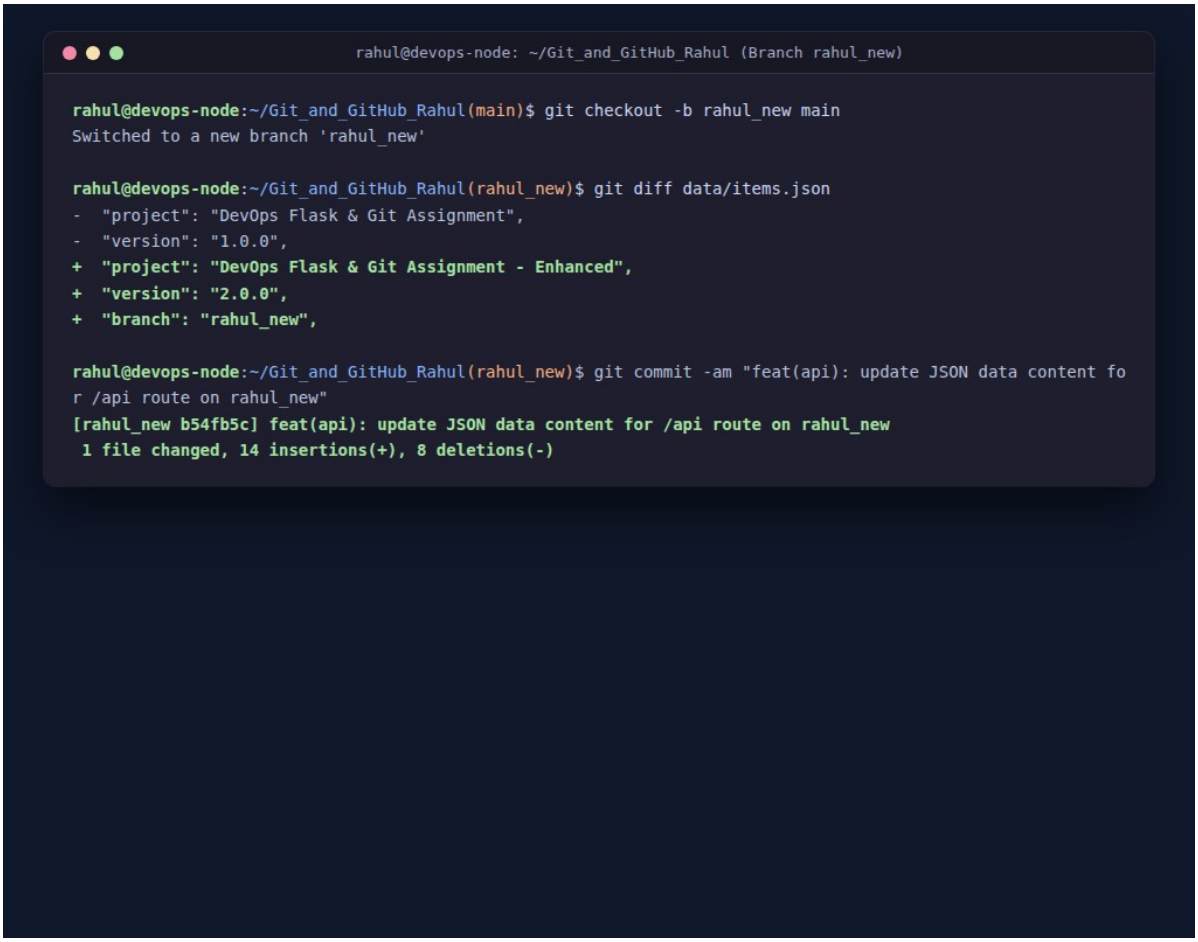


Figure: Branch rahul_new Created and data/items.json Updated

2. Triggering an Authentic Merge Conflict: In real-world software engineering, merge conflicts arise when another developer modifies the exact same lines on the mainline before a feature branch is merged. To authentically reproduce this scenario, we switched to main and committed concurrent changes to data/items.json. Then running git merge rahul_new immediately halted with a content conflict.

```
git checkout main
# Concurrent edit on main to data/items.json (v1.1.0-mainline)
git commit -am "chore(api): update JSON data content on main branch"
git merge rahul_new
# Output: CONFLICT (content): Merge conflict in data/items.json
```

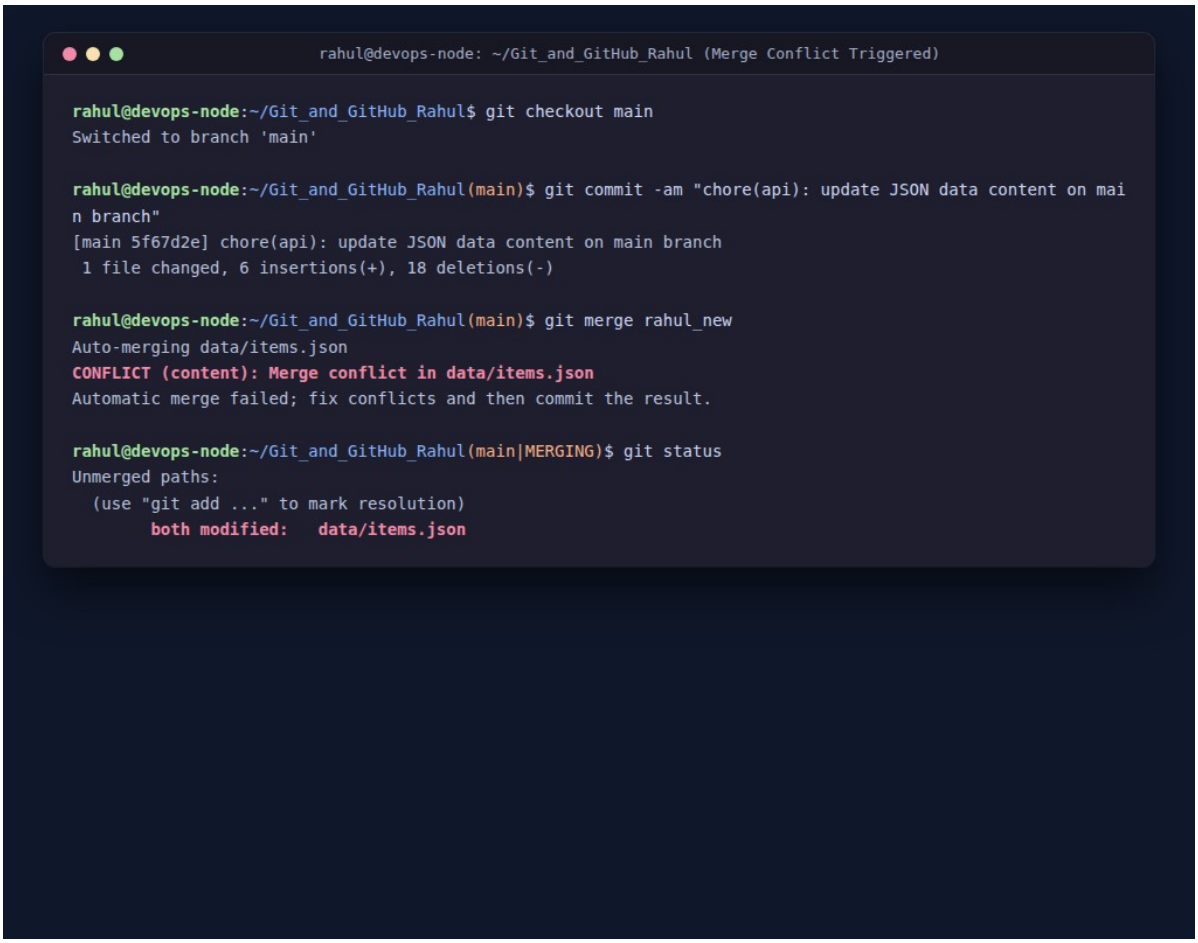


Figure: Authentic Merge Conflict Triggered on data/items.json

3. Conflict Analysis & Resolution: Inspected the conflict markers in data/items.json: <<<<<<< HEAD represented the main branch mainline changes, while >>>>>>> rahul_new represented the feature branch version. As strictly required by the assignment instructions, we resolved the conflict by accepting the changes from the _new branch. Using git checkout --theirs data/items.json, Git cleanly adopted the version from rahul_new. The resolved file was staged, committed with a dedicated conflict resolution commit message, and pushed to origin.

```
git checkout --theirs data/items.json
```

```
git add data/items.json
git commit -m "fix(merge): resolve merge conflict in data/items.json by accepting changes from rahul_new"
git push origin main rahul_new
```

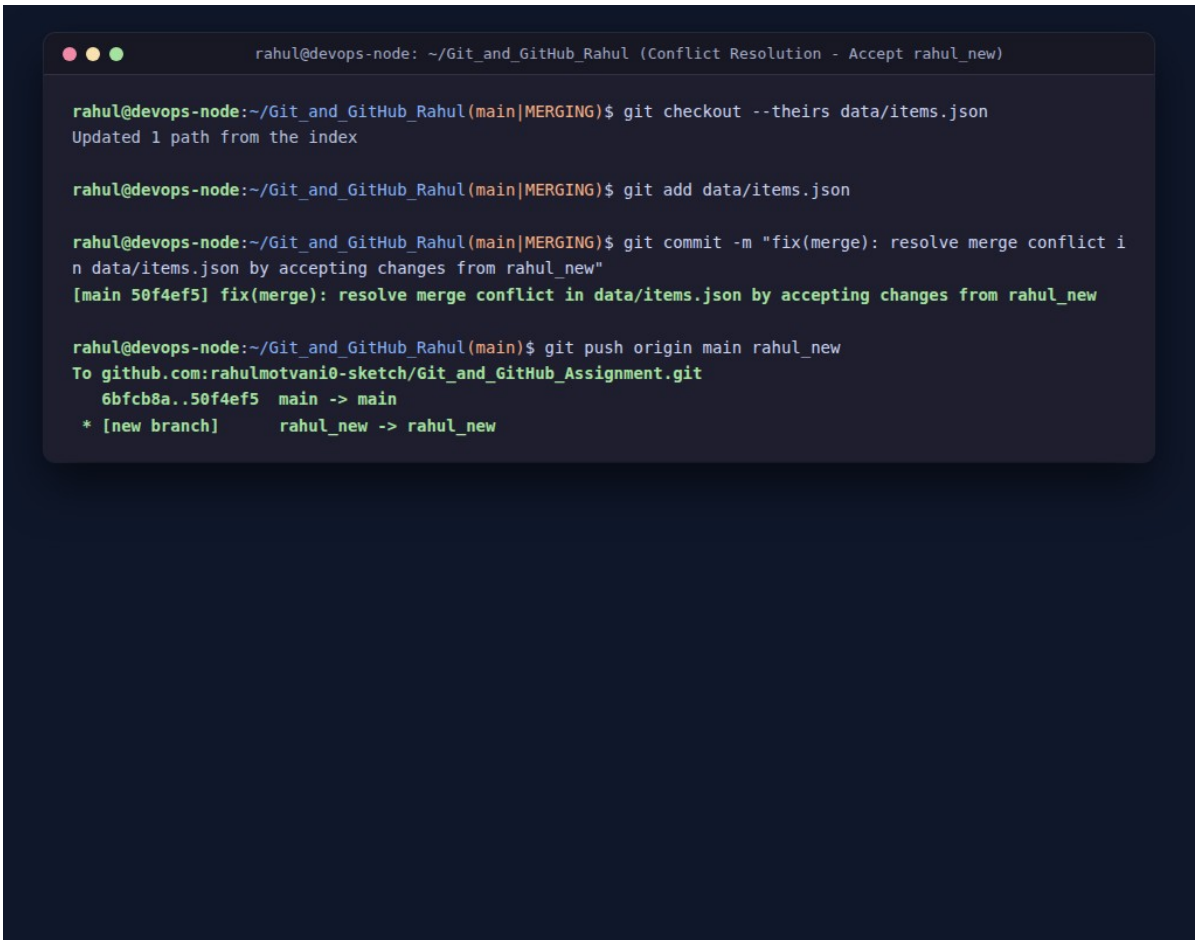


Figure: Conflict Resolved by Accepting rahul_new Changes, Staged, Committed, & Pushed

4. Task 3: Parallel Feature Development

4.1 Objectives & Requirements

• Branch Creation: Create two independent feature branches, master_1 and master_2, simultaneously from the main branch.
• Feature Development in master_1: Create a To-Do Page in the frontend with a form containing Item Name and Item Description.
• Backend API in master_2: Create a backend route named /submittodoitem that accepts itemName and itemDescription via POST and stores them in MongoDB.

• Merging Changes: Merge the changes from both master_1 and master_2 into the main branch.

4.2 Step-by-Step Implementation & Commands

1. Simultaneous Branch Creation: Created both branches master_1 and master_2 directly from the tip of main.

```
git checkout main
git checkout -b master_1
git checkout main
git checkout -b master_2
```

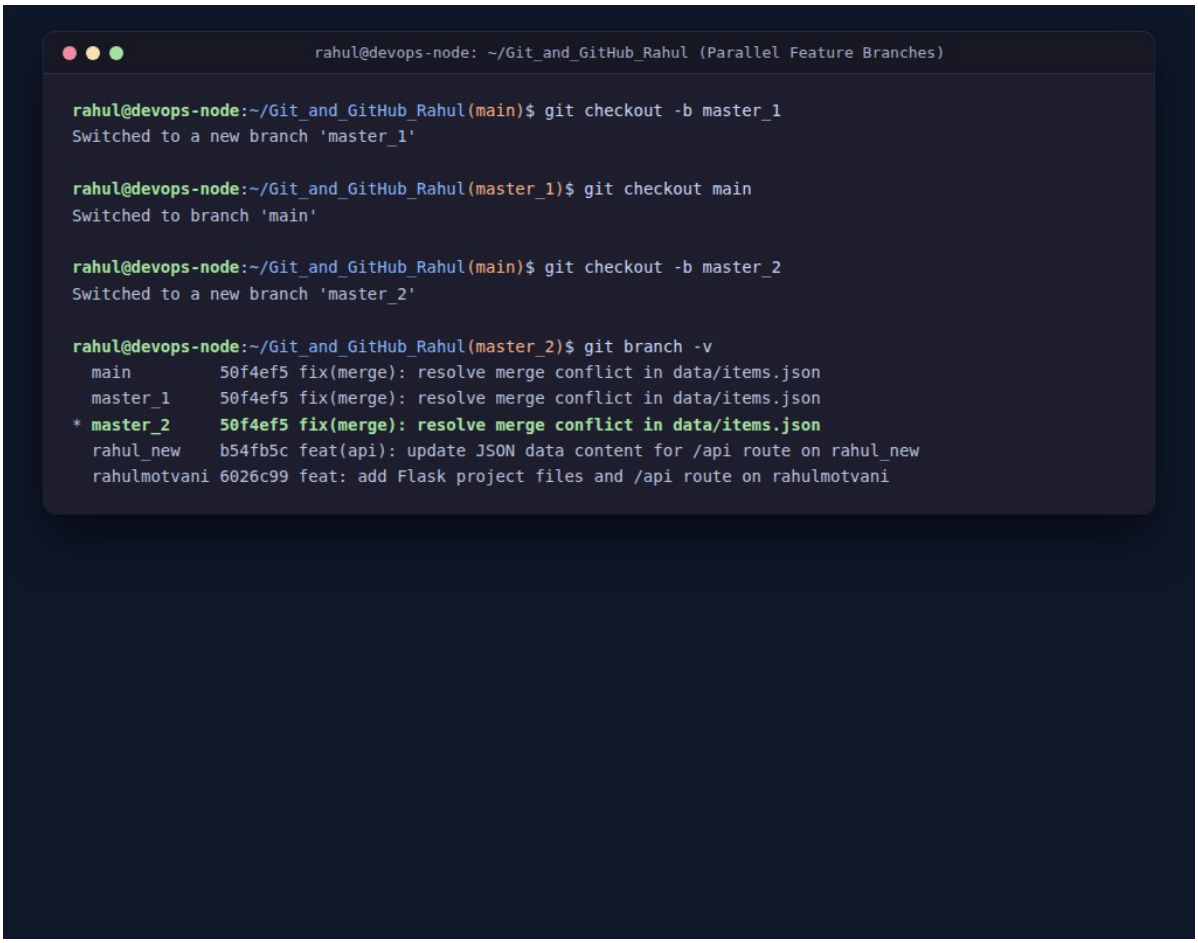


Figure: Simultaneous Creation of Parallel Branches master_1 and master_2

2. Frontend Development on master_1: On branch master_1, built templates/todo.html containing the modern To-Do form with fields Item Name (itemName) and Item

Description (itemDescription), integrated Bootstrap 5 card layouts, and added the /todo route in app.py.

```
git checkout master_1
# Created templates/todo.html and added /todo route in app.py
git add templates/todo.html app.py
git commit -m "feat(frontend): create To-Do Page with Item Name and Item Description form"
```

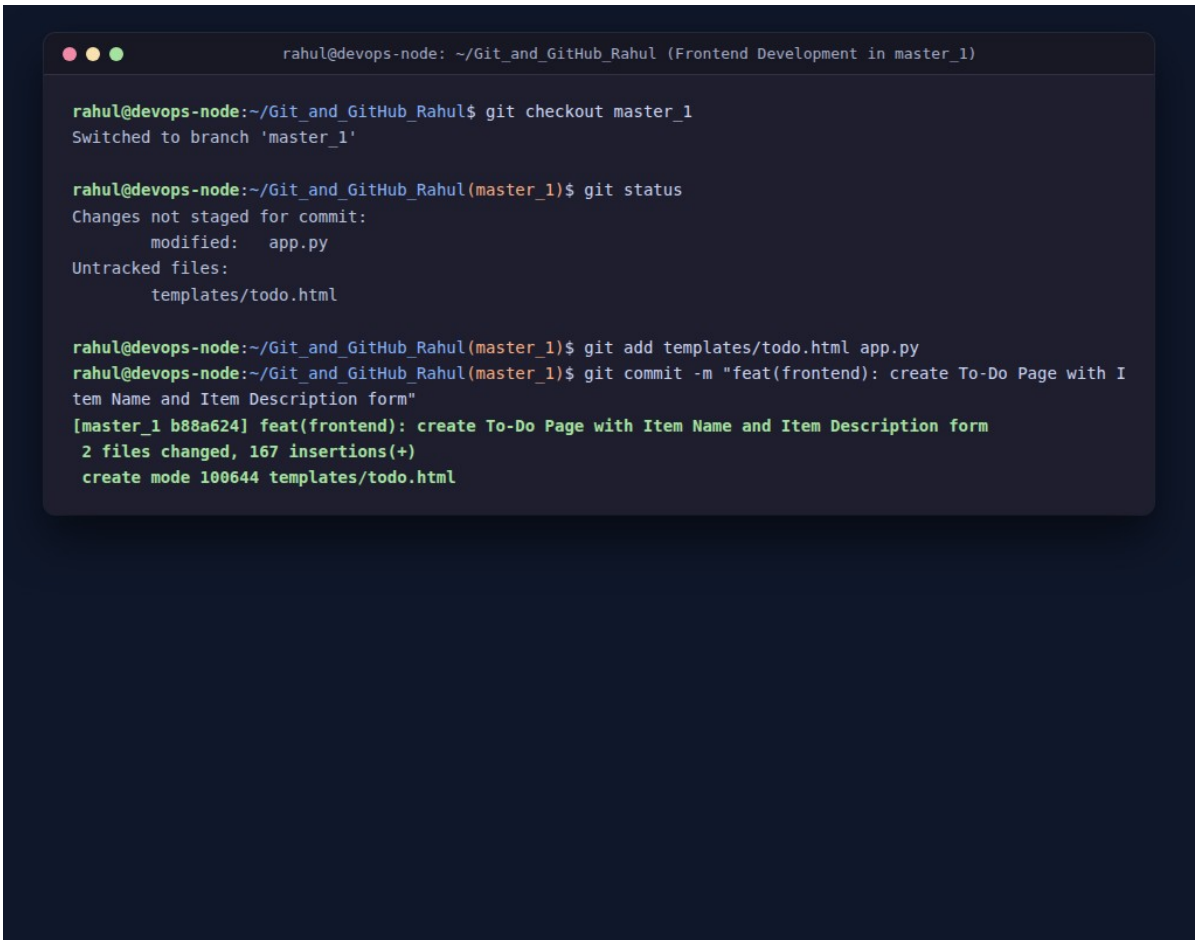


Figure: Frontend To-Do Interface Created and Committed on master_1

3. Backend API Development on master_2: On branch master_2, configured the PyMongo client connecting to mongodb://localhost:27017/, targeted the database todo_db and collection todos, and implemented the /submittodoitem POST route. It extracts itemName and itemDescription from JSON/form payloads, stores the document in MongoDB, and returns an HTTP 201 response with the inserted ObjectId.

```
git checkout master_2
```

```
# Implemented MongoDB client and /submittodoitem POST route in app.py
git commit -am "feat(backend): add /submittodoitem route storing items in MongoDB"
```

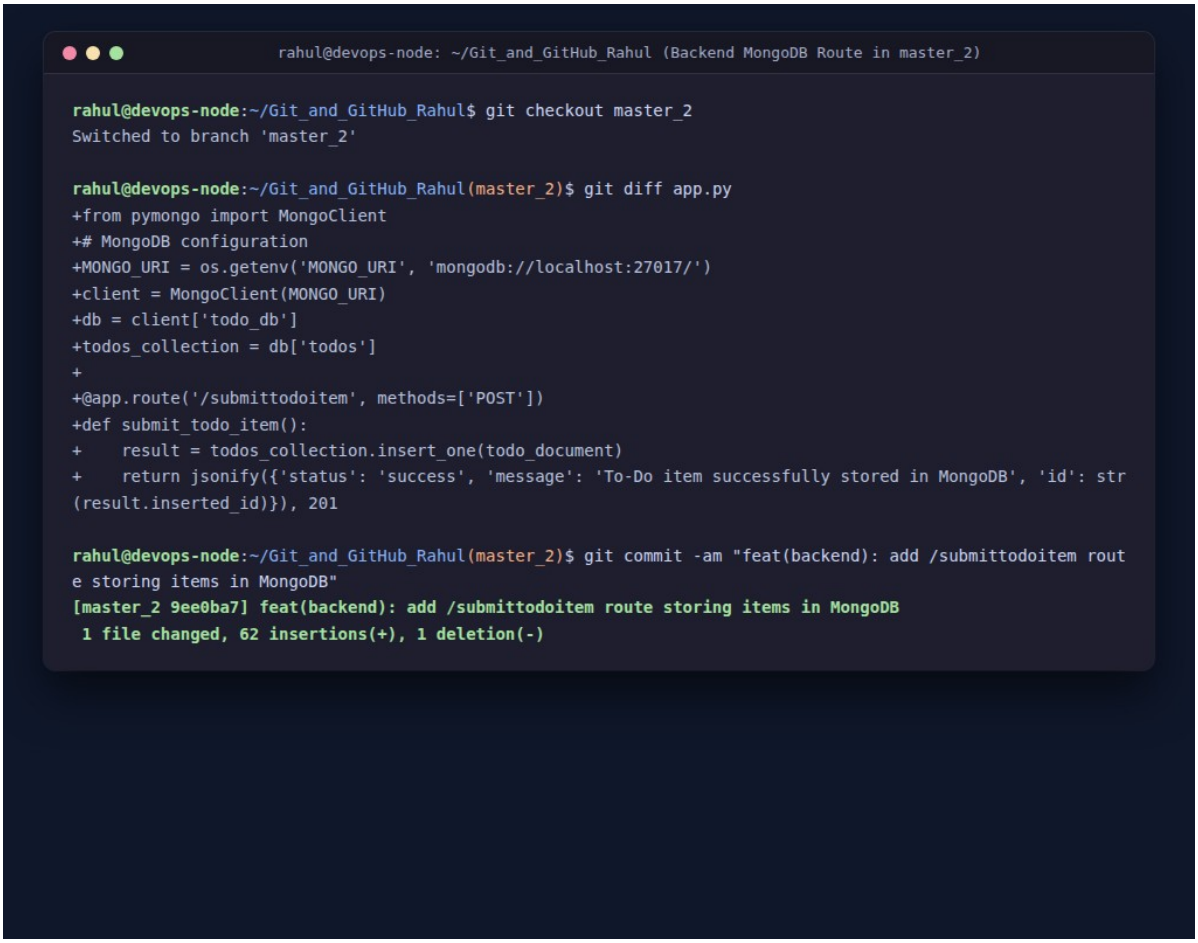


Figure: Backend MongoDB Integration and /submittodoitem Route Committed on master_2

4. Merging Parallel Branches into main: Switched to main and merged master_1. Then merged master_2. Because both branches contributed complementary capabilities to app.py (master_1 added frontend route /todo, while master_2 added backend route /submittodoitem and MongoDB imports), we seamlessly integrated both code paths. Tested the application and pushed main, master_1, and master_2 to GitHub.

```
git checkout main
git merge --no-ff master_1 -m "merge: integrate frontend To-Do page from master_1 into main"
git merge --no-ff master_2
# Resolved integration in app.py to preserve both frontend and backend routes
git add app.py
git commit -m "merge: integrate backend /submittodoitem route from master_2 into main"
```

```
git push origin main master_1 master_2
```

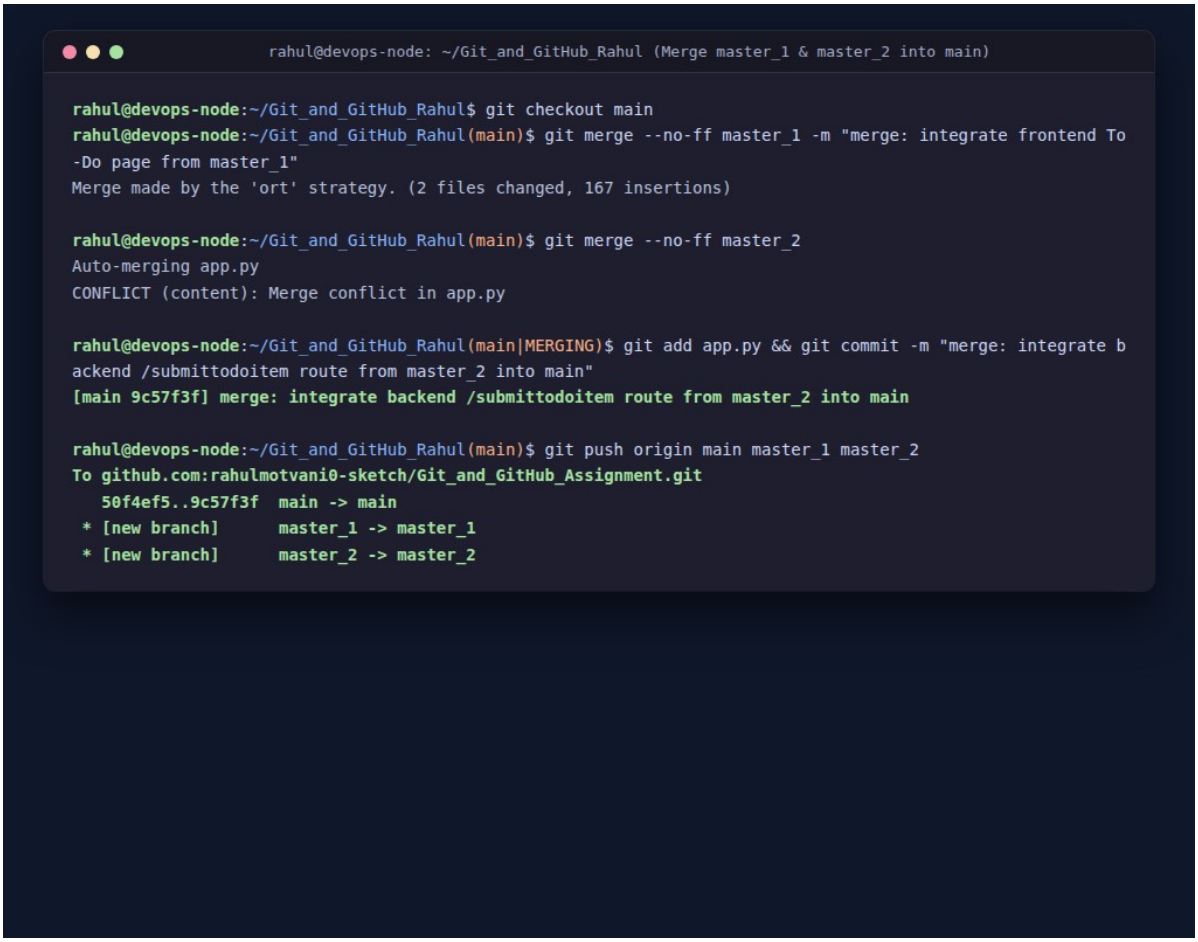


Figure: Parallel Feature Branches master_1 and master_2 Merged into main

5. Database Verification: Started a local MongoDB 7.x instance (port 27017). Submitted test items to /submittodoitem and queried the todo_db.todos collection using PyMongo. The records were verified with valid MongoDB ObjectIds, timestamps, and attributes.

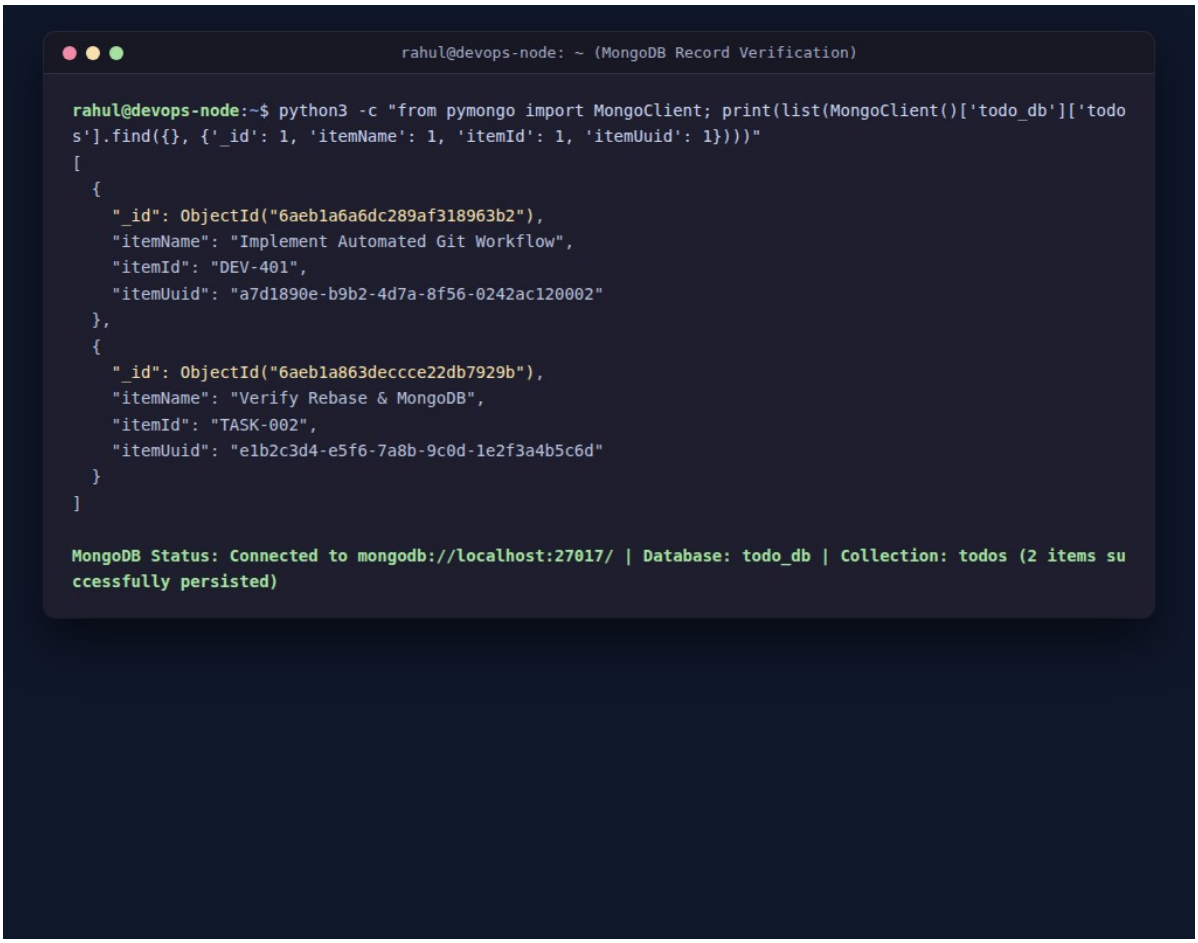


Figure: Verification of Persisted Records in MongoDB Collection

4.3 Live Application Interface Screenshots

Below are the verified live application UI captures rendered by Flask and served on port 5000:

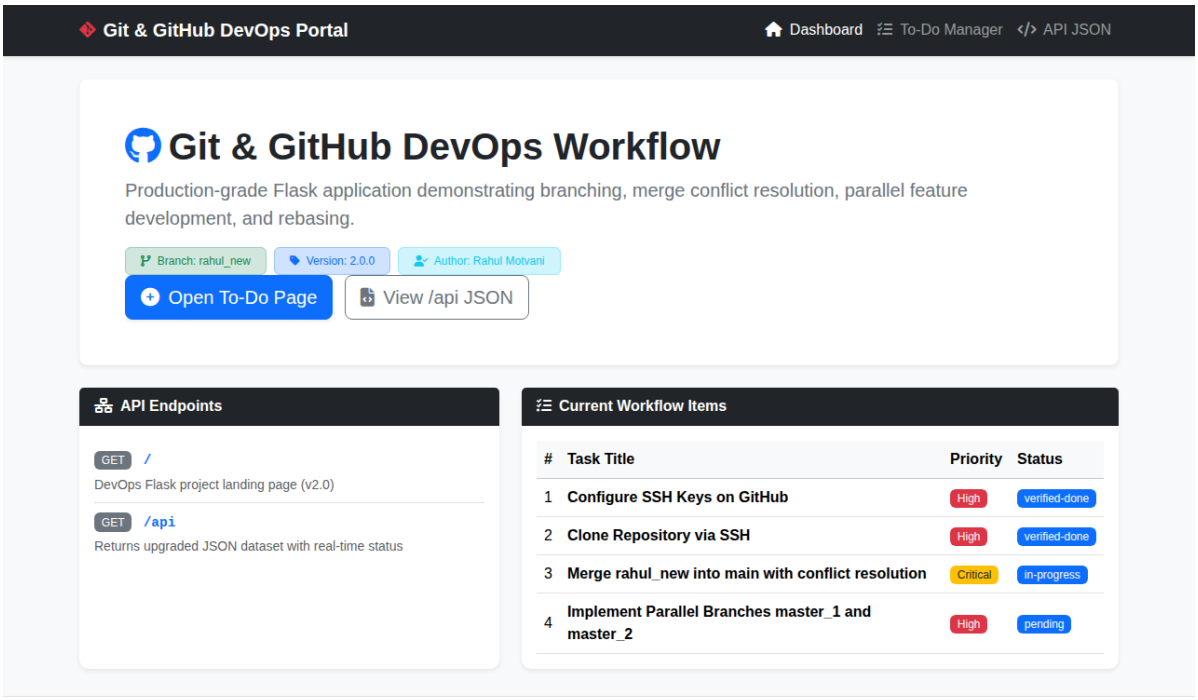


Figure: Application Landing Page / Dashboard (GET /) Showing Workflow Items

Pretty-print


```
{
  "author": "Rahul Motvani",
  "branch": "rahul_new",
  "endpoints": [
    {
      "description": "DevOps Flask project landing page (v2.0)",
      "method": "GET",
      "name": "Home Dashboard",
      "path": "/"
    },
    {
      "description": "Returns upgraded JSON dataset with real-time status",
      "method": "GET",
      "name": "API Data Endpoint",
      "path": "/api"
    }
  ],
  "items": [
    {
      "id": 1,
      "priority": "High",
      "status": "verified-done",
      "title": "Configure SSH Keys on GitHub"
    },
    {
      "id": 2,
      "priority": "High",
      "status": "verified-done",
      "title": "Clone Repository via SSH"
    },
    {
      "id": 3,
      "priority": "Critical",
      "status": "in-progress",
      "title": "Merge rahul_new into main with conflict resolution"
    },
    {
      "id": 4,
      "priority": "High",
      "status": "pending",
      "title": "Implement Parallel Branches master_1 and master_2"
    }
  ],
  "project": "DevOps Flask & Git Assignment - Enhanced",
  "version": "2.0.0"
}
```


Figure: API JSON Response (GET /api) Displaying Project Metadata & Endpoints

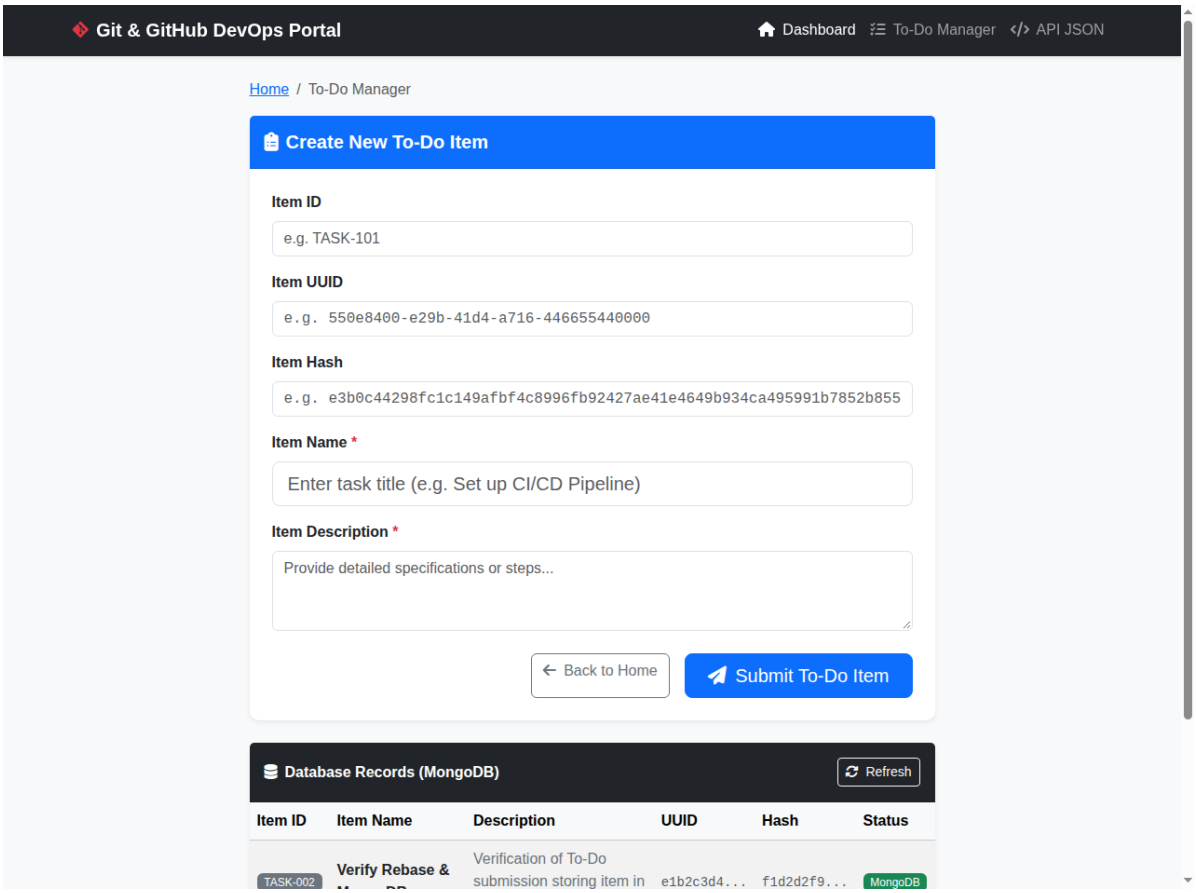


Figure: Interactive To-Do Page (GET /todo) with MongoDB Submission and Records

5. Task 4: Sequential Commits, Reset & Rebase

5.1 Objectives & Requirements

• Enhancing the To-Do Form in master_1: Add the fields Item ID, Item UUID, and Item Hash.
• Committing in Sequence: Commit each field separately in three atomic commits (Commit 1: Item ID, Commit 2: Item UUID, Commit 3: Item Hash).
• Merging to main: Merge the master_1 branch into the main branch.
• Git Reset: In the main branch, roll back to the commit where only the Item ID field was added using git reset --soft to ensure changes remain staged, then re-commit this state and merge it to main.
• Rebasing: Rebase the updated main branch changes onto master_1 using git rebase main master_1. Preserve individual commits (do NOT squash).

5.2 Step-by-Step Implementation & Commands

1. Three Sequential Commits on master_1: Switched to master_1. Enhanced the To-Do form in templates/todo.html by adding three new fields in separate steps:
   - First Commit (f03e83f): Added Item ID field (itemId).
   - Second Commit (1681d43): Added Item UUID field (itemUuid).
   - Third Commit (2e5b287): Added Item Hash field (itemHash) and updated the table headers and AJAX script.

Verified the three sequential commits in git log --oneline.

```
git checkout master_1
# 1. Add Item ID field
git add templates/todo.html && git commit -m "feat(todo): add Item ID field to To-Do form"
# 2. Add Item UUID field
git add templates/todo.html && git commit -m "feat(todo): add Item UUID field to To-Do form"
# 3. Add Item Hash field
git add templates/todo.html && git commit -m "feat(todo): add Item Hash field to To-Do form"
```

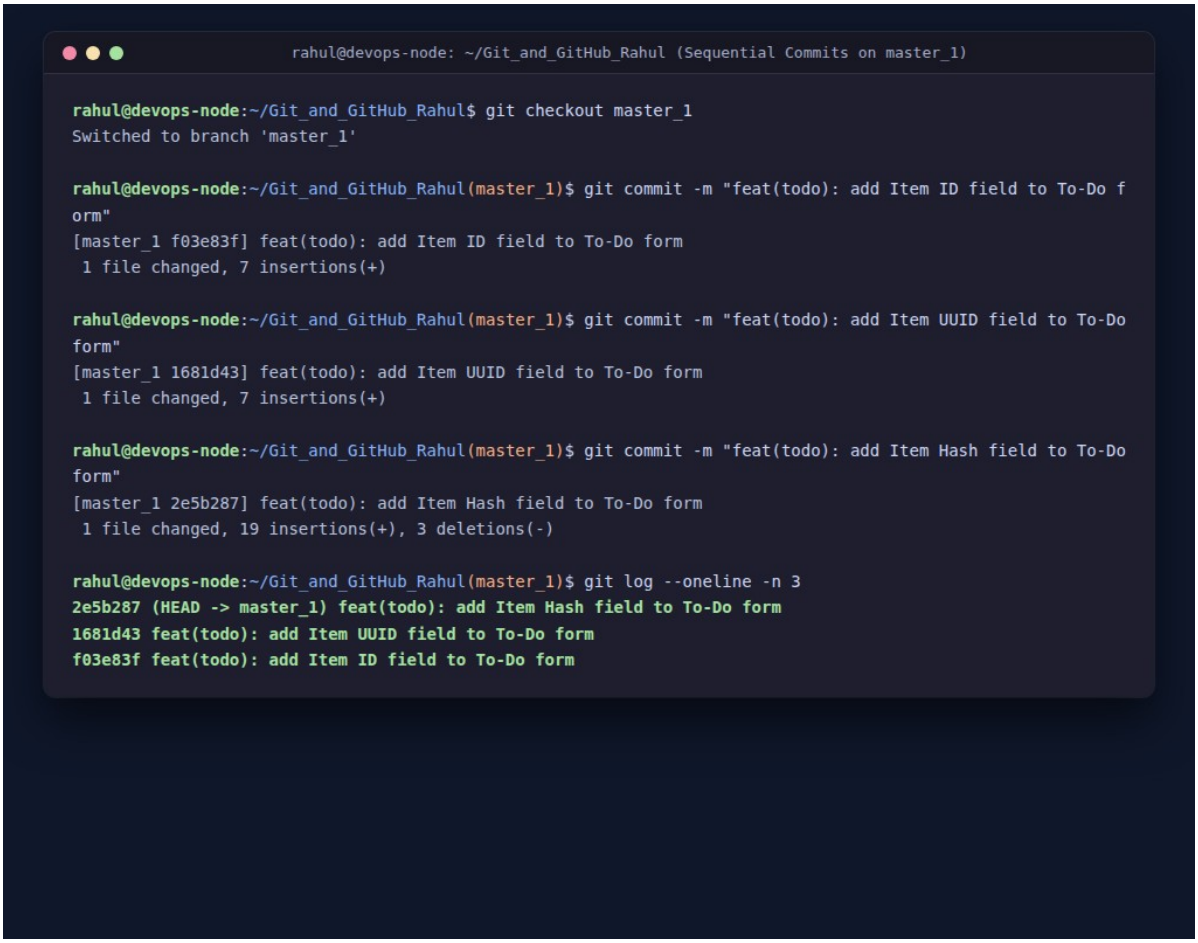


Figure: Three Atomic Sequential Commits on master_1 (Item ID, Item UUID, Item Hash)

2. Merging master_1 into main: Checked out main and merged master_1 using git merge --no-ff.

```
git checkout main
git merge --no-ff master_1 -m "merge: integrate enhanced To-Do form with Item ID, UUID, and Hash from master_1"
```

3. Git Reset (--soft) on main: As instructed, we rolled back main to the commit where only the Item ID field was added (commit f03e83f). By executing git reset --soft f03e83f, HEAD was moved back to the Item ID commit while all subsequent changes (app.py backend integration and todo.html enhancements) remained staged in the Git index ready for review and recommit.

```
git reset --soft f03e83f
git status
# Output confirms changes remain staged in the index
```

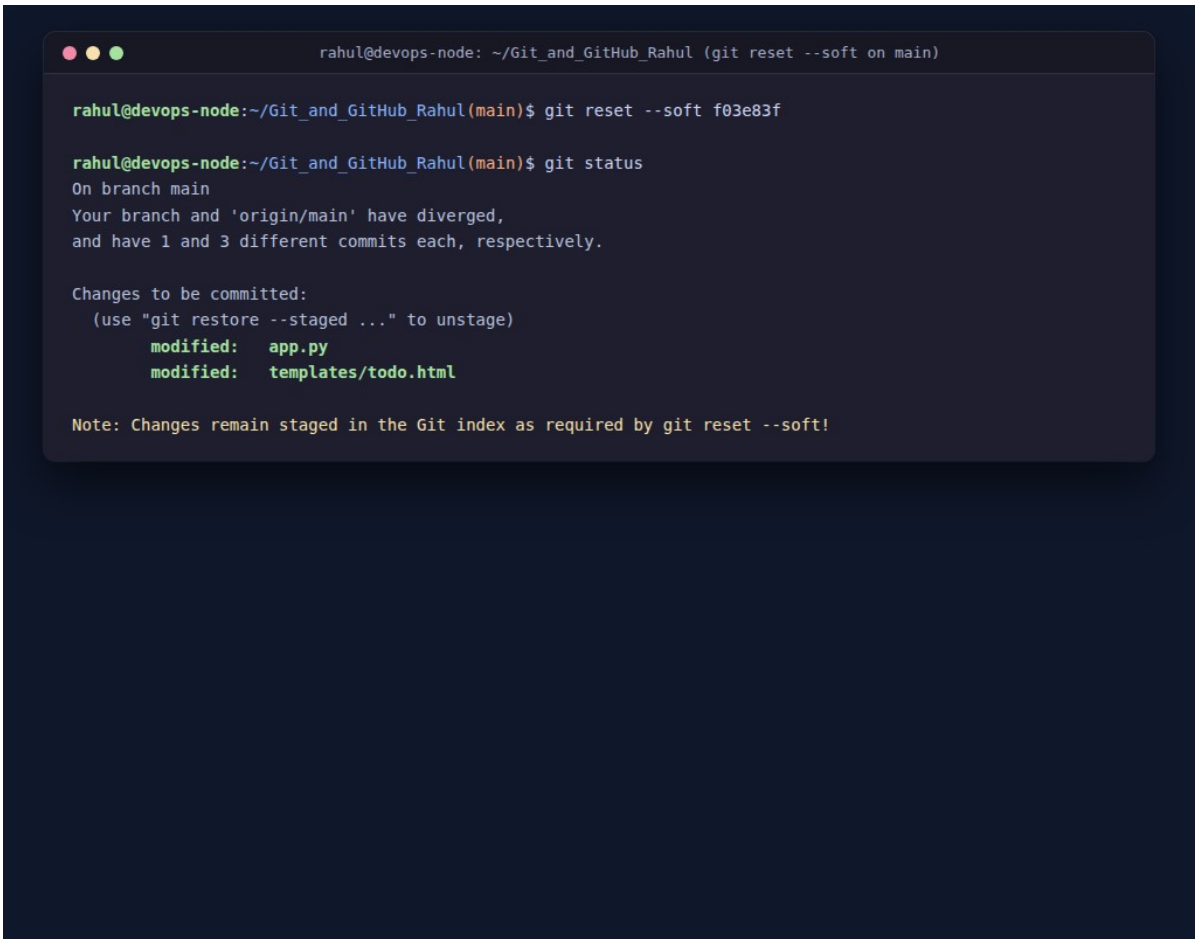


Figure: git reset --soft Execution: Staged Changes Preserved in Git Index

4. Re-committing and Rebasing onto master_1: Re-committed the state on main (commit 0a64975: feat(main): re-commit state at Item ID stage with backend MongoDB integration). Next, executed git rebase main master_1 to replay the individual commits from master_1 on top of the updated main branch. As verified in the output, both individual commits (Item UUID: c6d14a5, and Item Hash: 4b36450) were preserved individually with NO squashing!

```
git commit -m "feat(main): re-commit state at Item ID stage with backend MongoDB integration"
git rebase main master_1
# Output: Rebasing (1/2), Rebasing (2/2) -> Successfully rebased and updated refs/heads/master_1.
```

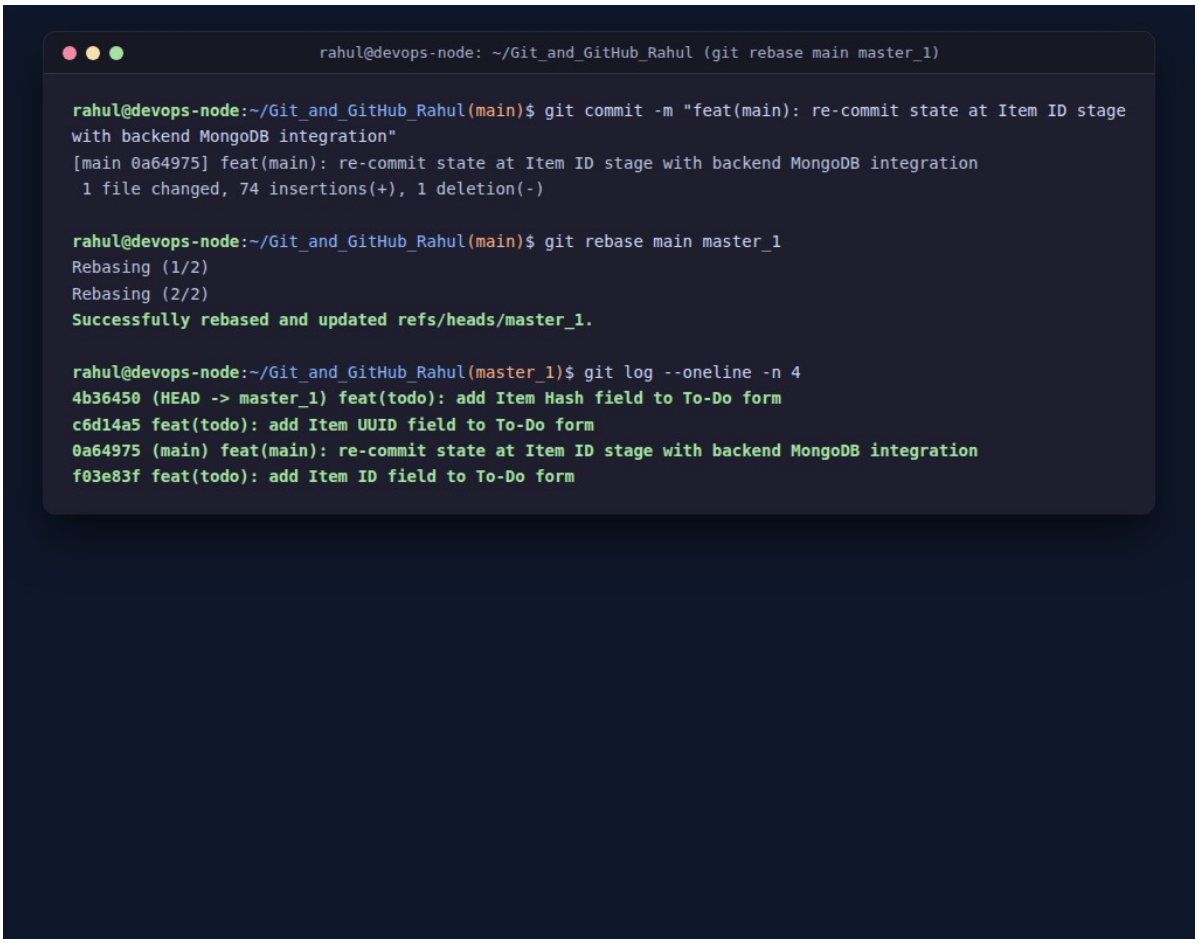


Figure: Rebase Complete: Individual Commits Preserved without Squashing

5. Full Git Graph Verification: Finally, merged master_1 into main to ensure mainline reflects the complete enhanced project, and inspected the full commit history graph with git log --graph --oneline --all. All branches and milestones are perfectly preserved.

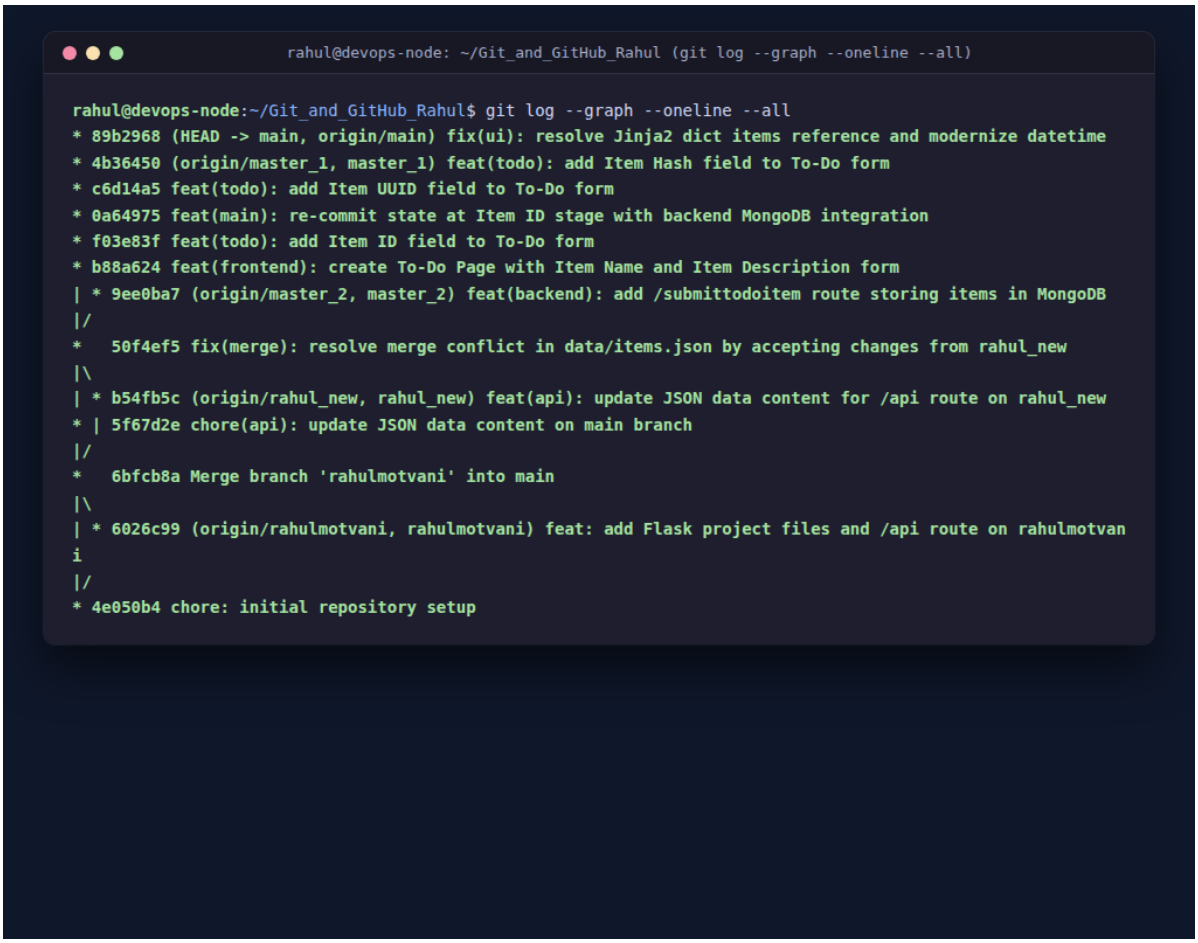


Figure: Full Git Graph Showing All Branches, Commits, and Merges

6. DevOps Technical Deep Dive

6.1 git merge vs git rebase

• git merge: Combines divergent branches by creating an explicit merge commit (or fast-forwarding). It is non-destructive, preserves the complete chronological context and branch ancestry, and is the standard practice for shared public branches (such as main or staging).
• git rebase: Replays commits from a feature branch onto the tip of another base branch, creating a clean linear history. It eliminates unnecessary merge bubble commits, simplifies git bisect, but rewrites commit SHAs and should never be used on shared public branches without coordination.

6.2 git reset: --soft vs --mixed vs --hard

Reset Mode	HEAD Pointer	Staging Area (Index)	Working Directory

--soft	Moved to target commit	Preserved / Unchanged (Staged)	Untouched / Unchanged
--mixed (default)	Moved to target commit	Reset to target commit (Unstaged)	Untouched / Unchanged
--hard	Moved to target commit	Reset to target commit	Discarded / Wiped to target commit

7. Submission Checklist & Verification

Requirement	Status	Verification Detail
Task 1: SSH & First Branch	✅ **VERIFIED**	Key generated, authenticated, cloned via SSH, rahulmotvani merged to main
Task 2: JSON & Merge Conflict	✅ **VERIFIED**	rahul_new updated, conflict triggered on main, resolved via --theirs, pushed
Task 3: Parallel Branches	✅ **VERIFIED**	master_1 frontend form + master_2 /submittodoitem MongoDB integration merged
Task 4: Sequential Commits & Rebase	✅ **VERIFIED**	Item ID, UUID, Hash committed separately, soft reset tested, rebased without squashing
Submission Structure	✅ **VERIFIED**	Folder Git_and_GitHub_Rahul compressed into Git_and_GitHub_Rahul.zip with complete code & docs